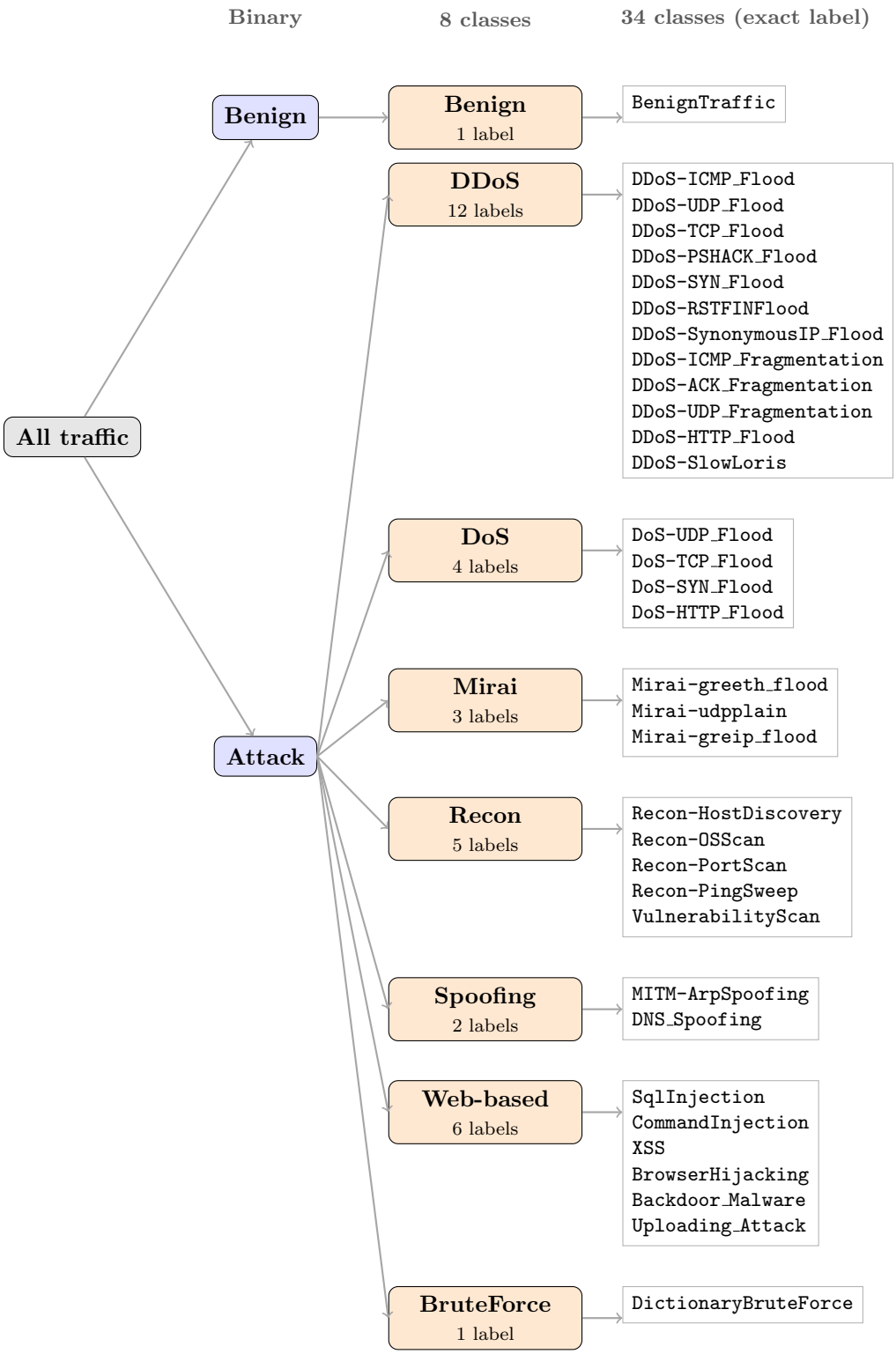
CICIoT2023 labels

2-class, 8-class and 34-class: one tree

Same rows every time, only the *answer* the model must give changes.
Example row labelled `DDoS-SYN_Flood`: binary → **Attack**, 8-class → **DDoS**, 34-class → **DDoS-SYN_Flood**.



How the data was captured

The lab

The Canadian Institute for Cybersecurity built a lab network with **105 real IoT devices** (cameras, smart plugs, speakers, sensors, hubs ...). Some extra machines (Raspberry Pis) were set up as **attackers**: they attacked the IoT devices while everything else kept running normally. A switch copied ("mirrored") **every packet on the network** to a capture computer, which saved them into `.pcap` files (Wireshark format).

Is this MQTT or CoAP data?

No, not specifically. The capture is *all* network traffic: TCP, UDP, ICMP, ARP, DNS, web, everything. MQTT is just TCP traffic on port 1883 and CoAP is just UDP traffic on port 5683, so they are *inside* the data as ordinary TCP/UDP packets. There is no MQTT or CoAP column.

What is the "window"? Is it 1 second?

No. The window is a number of packets, not a time. The authors computed the features for each packet, then **averaged them over a fixed number of consecutive packets** Each row of the CSV is one such window. The paper (reference 1 of the TinyIDS PDF) gives the exact sizes:

- **100 packets** per row for all **DDoS, DoS and Mirai** attacks (floods produce huge amounts of packets).
- **10 packets** per row for **Benign, Recon, Spoofing, Web-based and BruteForce**.

Consequence: the real time covered by a row changes a lot. During a flood, 100 packets arrive in about a millisecond. In normal quiet traffic, 10 packets can take several seconds. So you cannot say "this row = 1 second of traffic". This is the same problem you had with CICDDoS2019: if you want "packets per second towards a victim", you have to compute it yourself on live traffic.

"Incoming" and "outgoing": from where?

Direction is seen from the **sender of the packets being measured**: outgoing = packets it sends, incoming = packets coming back to it. In practice this did not work well in CICIoT2023. I checked your `test.csv`:

- `Srate` (outgoing rate) is **exactly equal** to `Rate` in all 1,176,851 rows.
- `Drate` (incoming rate) is non-zero in only **74 rows** (0.006%).

So `Srate` and `Drate` add nothing: **drop them and just use** `Rate`.

From packets to one row: how a SYN flood is detected

"If there is no time column, where does Rate come from?"

Every captured packet carries a **timestamp**: the exact moment it arrived, to the microsecond (Wireshark shows it; in scapy it is `pkt.time`). The authors *had* these timestamps and used them to calculate `Rate`, `flow_duration` and `IAT`. Then they deleted the timestamp column from the CSV (the paper says it was only used for sorting). So **time went into the calculation; only the results were kept.**

Worked example: one SYN-flood row (window = 100 packets)

Packet	Arrival time (s)	Flags	Size (bytes)
1	12.000000	SYN	54
2	12.000010	SYN	54
3	12.000020	SYN	54
...	...	...	...
100	12.001000	SYN	54

`flow_duration`	last time − first time = 12.001000 − 12.000000 = **0.001 s**
`Rate`	packets ÷ duration = 100 ÷ 0.001 = **100,000 packets/s**
`IAT`	average gap between packets ≈ **0.00001 s**
`syn_flag_number`	100 SYN out of 100 = **1.0**
`ack_flag_number`	0 out of 100 = **0.0** (nobody completes the handshake)
`TCP`	**1.0**
`Tot size`	**54 bytes**, all identical, so `Std` ≈ **0**

Model sees: huge rate + 100% SYN + 0% ACK + all TCP + tiny identical packets ⇒ **SYN flood**.

Compare: normal web browsing (window = 10 packets)

10 packets spread over about 2 seconds: `Rate` ≈ **5 packets/s**; a mix of SYN, ACK and PSH flags; packet sizes from 54 to 1500 bytes, so `Std` is large. ⇒ **Benign**.

And someone retrying a dead website (10 SYNs in 5 minutes): `Rate` = 10 ÷ 300 ≈ **0.03 packets/s**. 100% SYN, but nowhere near flood speed ⇒ does not look like the flood rows.

Doing the same thing live on the Raspberry Pi

This is the "Flow feature extractor" box in the TinyIDS diagram: sniff packets, keep the last N, compute the same numbers, ask the model.

```
from scapy.all import sniff, TCP
N = 100          # 100 to match the flood rows in the dataset
buf = []

def on_packet(pkt):
    buf.append(pkt)
    if len(buf) == N:
```

```
        t = [float(p.time) for p in buf]
        duration = t[-1] - t[0]
        rate = N / duration if duration > 0 else 1e6
        syn  = sum(1 for p in buf if TCP in p and p[TCP].flags.S) / N
        ack  = sum(1 for p in buf if TCP in p and p[TCP].flags.A) / N
        tcp  = sum(1 for p in buf if TCP in p) / N
        size = sum(len(p) for p in buf) / N
        # ... same for the other features you keep ...
        # label = model.predict([[duration, rate, syn, ack, tcp, size, ...]])
        buf.clear()

sniff(iface="wlan0", prn=on_packet, store=False)
```

Caveat: live numbers will not match the dataset exactly

- The paper **does not publish the formulas** for most features and does not link its extraction code (see the next section). My formulas above are the obvious, sensible ones, but the authors' versions may differ in details.
- The dataset also mixes in flow information (e.g. `syn_count` = SYNs "in the same flow"), which a simple sliding window does not reproduce exactly.
- Your testbed is different hardware and a different network from their lab.

What to do about it:

1. Train only on the **10–15 features you can compute yourself** (Rate, flow_duration, IAT, flag fractions, protocol fractions, packet size stats). Drop Magnitue, Radius, Covariance, Variance, Weight, Srate, Drate.
2. Don't alert on a single window: alert when most windows in the last 1–2 seconds are flagged.
3. In the report, say that live testbed results are a **demonstration**, not the main accuracy figure. The TinyIDS PDF itself says this in its "Validity note".

What formulas do the authors actually use?

Short answer: mostly, they don't say.

The CICIoT2023 paper (Neto et al., *Sensors* 2023, Table 4) gives:

- **Exact formulas for only 5 features:** `Magnitue`, `Radius`, `Covariance`, `Variance` and `Weight`. These are the "incoming vs outgoing" ones.
- **Standard definitions** for the packet-size stats (`Tot sum`, `Min`, `Max`, `AVG`, `Std`) and the flag columns.
- **Only one-line descriptions, no formula,** for `Rate`, `flow_duration` and `IAT`. For example, `Rate` is just described as "rate of packet transmission in a flow".
- **Their tools:** `tcpdump` to split the captures, the Python library `dpkt` to read packets, and pandas/NumPy to average each window.
- **The paper doesn't link any extraction code.** The only link it gives is to the dataset download page.

Source: `https://pmc.ncbi.nlm.nih.gov/articles/PMC10346235/`

Why this matters for you: because the exact recipe for `Rate`, `flow_duration` and `IAT` is not published, nobody can reproduce those columns exactly. When you compute them live on the Pi you use the natural definitions (packets ÷ time, last time − first time, average gap), and your numbers will be close but not identical to the dataset's.

Feature by feature

Feature	What the paper gives
`Magnitue`	**Formula:** $\sqrt{\text{avg incoming length} + \text{avg outgoing length}}$
`Radius`	**Formula:** $\sqrt{\text{var incoming length} + \text{var outgoing length}}$
`Covariance`	**Formula:** covariance of incoming and outgoing packet lengths
`Variance`	**Formula:** var(incoming lengths) ÷ var(outgoing lengths)
`Weight`	**Formula:** # incoming packets × # outgoing packets
`Tot sum, Min, Max, AVG, Std`	Sum / min / max / mean / standard deviation of packet lengths in the flow (standard definitions).
`Number`	Number of packets in the flow.
`flag numbers`	Value (0/1) of that TCP flag in the packet, then averaged over the window.
`flag counts`	Number of packets with that flag in the same flow.
`Rate, Srate, Drate`	**Description only**, no formula: "rate of packet transmission in a flow" (all / outbound / inbound).
`flow_duration`	**Description only**: "duration of the packet's flow".
`IAT`	**Description only**: "time difference with the previous packet".
`Duration`	Time-to-live (TTL) of the packet.

Bottom line: only the five "incoming vs outgoing" features have exact formulas. For Rate, duration and IAT you must use the natural definitions, as in the example above.

The attacks explained

First: DoS vs DDoS

A **DoS** (Denial of Service) attack makes a device or service unusable by overloading it, usually by sending far more traffic than it can handle.

- **DoS** = the flood comes from **one** attacker machine.
- **DDoS** (Distributed DoS) = the flood comes from **many** machines at once. Much harder to block because you cannot just ban one IP.

In the lab, DoS used one attacking Raspberry Pi and DDoS used many of them together. The flood *types* are the same, which is why both families have SYN, UDP, TCP and HTTP floods.

Background you need: the TCP handshake

Before two machines exchange data over TCP they do a 3-step handshake: client sends **SYN** ("can we talk?") → server replies **SYN-ACK** ("yes, I reserved space for you") → client sends **ACK** ("great, let's go"). Many floods abuse this.

DDoS family (12 labels) and DoS family (4 labels)

SYN_Flood The attacker sends a huge number of SYN packets, usually with fake source IPs, and never sends the final ACK. For each SYN, the server reserves memory and waits (a "half-open" connection). Its waiting list fills up, and real users can no longer connect. Typical tool: `hping3 -S --flood`. *This is the attack in your thesis.*

SynonymousIP_Flood A SYN flood where the fake source IP is set to the *victim's own IP*. The victim ends up trying to reply to itself, which confuses some network stacks and wastes more resources.

TCP_Flood A flood of generic TCP packets (not just SYN) at high speed, to eat bandwidth and CPU.

PSHACK_Flood Packets with the PSH and ACK flags set, pretending to be data in an already open connection. The server has to look each one up in its connection table before throwing it away. It also slips past defences that only watch for SYNs.

RSTFINFlood Packets with RST ("abort") or FIN ("close") flags. Again the server must check each against its connection table, costing CPU.

UDP_Flood Massive numbers of UDP packets to random ports. UDP needs no handshake, so it is cheap to send. For each packet the victim checks whether any program listens on that port and often replies "port unreachable" (ICMP), wasting even more bandwidth.

ICMP_Flood Also called a ping flood: nonstop ping (ICMP echo request) packets. The victim's link fills up and it spends CPU replying to each ping.

ICMP / UDP / ACK_Fragmentation Big packets are split into small pieces (fragments) that the receiver must collect and reassemble. The attacker sends fragments that are broken or never complete. The victim keeps them in memory waiting for the missing pieces, and runs out of memory/CPU. The three labels differ only in which packet type is fragmented.

HTTP_Flood Lots of normal-looking web requests ("GET /index.html") sent to a web server. Each one makes the server do real work (read files, run code). Hard to filter because every single request looks legitimate.

SlowLoris A "slow" attack: the attacker opens many connections to a web server and sends the request headers *very slowly*, a few bytes every few seconds, never finishing. The server keeps

each connection open waiting, and soon all its connection slots are taken. Needs almost no bandwidth.

DoS labels (`DoS-UDP_Flood`, `DoS-TCP_Flood`, `DoS-SYN_Flood`, `DoS-HTTP_Flood`) are the same attacks as above, from a single machine.

Mirai family (3 labels)

What Mirai is. Mirai is real malware from 2016. It scans the internet for IoT devices (cameras, routers, DVRs) that have Telnet open, and tries to log in with a list of about 60 **factory-default passwords** like `admin/admin` or `root/12345`. When it gets in, it infects the device, which becomes a **bot**. The bots wait for commands from the attacker's control server and then all flood a target together. In October 2016 a Mirai botnet of hundreds of thousands of cameras and routers took down the DNS provider Dyn, which made Twitter, Netflix and Reddit unreachable for hours. The source code was leaked, so many copies exist today. This is exactly why IoT security matters.

Mirai-greeth_flood Floods GRE packets. GRE is a "tunnel" protocol that wraps one packet inside another. Here each GRE packet carries a fake **Ethernet** frame, so the victim must unwrap every one.

Mirai-greip_flood Same idea, but each GRE packet carries a fake **IP** packet.

Mirai-udpplain A plain UDP flood tuned for maximum packets per second (fewer random fields, so the bots can send faster).

Recon family (5 labels)

Recon (reconnaissance) means scouting before an attack: what devices exist, what they run, and where the weak spots are. It does not damage anything by itself. Typical tool: `nmap`.

Recon-HostDiscovery Find which IP addresses on the network are alive (e.g. `nmap -sn`).

Recon-PingSweep A simple form of host discovery: ping every address in a range and see who answers.

Recon-PortScan For one device, try many ports to see which are open. An open port answers SYN with SYN-ACK; a closed port answers with RST. That is why RST counts rise during scans. *This is in your live demo.*

Recon-OSScan Send unusual probe packets and look at *how* the device answers (TTL, TCP window size, option order). Each operating system answers slightly differently, which reveals whether it runs Linux, Windows, an embedded OS, etc.

VulnerabilityScan Automated tools (e.g. OpenVAS, Nessus) check each open service against a database of known security holes.

Spoofing family (2 labels)

MITM-ArpSpoofing On a local network, devices use ARP to ask "who has IP 192.168.1.1 (the router)?". The attacker keeps sending fake answers: "that's me". Victims then send their traffic to the attacker, who forwards it on, so nobody notices. The attacker sits in the middle (MITM = man-in-the-middle) and can read or change everything.

DNS_Spoofing When a device asks DNS "what is the IP of example.com?", the attacker answers first with a fake IP. The device then connects to the attacker's server instead of the real one.

Web-based family (6 labels)

These target web applications, e.g. a camera's or router's admin web page.

SqlInjection The attacker types database code into a form field. Example: as the password, entering `' OR '1'='1`. If the site builds its database query by pasting in the input, the check becomes "always true" and the attacker logs in, or dumps the whole database.

CommandInjection Same idea, but the input is passed to the operating system's shell. Example: a "ping this IP" box where the attacker enters `8.8.8.8; cat /etc/passwd`, and the device runs both commands.

XSS Cross-site scripting: the attacker stores JavaScript inside a page (e.g. in a comment or device name). When someone else opens that page, the script runs in *their* browser and can steal their login cookie.

BrowserHijacking Malicious code changes the victim's browser behaviour: redirects to other sites, changes the home page, injects ads, captures what they type.

Backdoor_Malware Installing a hidden program on the device that lets the attacker come back later with remote access, bypassing the normal login.

Uploading_Attack Abusing a file-upload feature (e.g. "upload firmware" or "upload picture") to upload a malicious script disguised as a normal file. If the server runs it, the attacker controls the server.

BruteForce family (1 label)

DictionaryBruteForce Try to log in again and again using a list of common passwords (a "dictionary" such as `rockyou.txt`) until one works. Usually against Telnet, SSH or a web login. The **MQTT brute force** in the TinyIDS demo is this same attack against the MQTT broker's username/password.

Benign

BenignTraffic Normal traffic from the 105 devices with no attack running: cameras streaming, plugs reporting status, hubs syncing, DNS lookups, updates, etc.

The 8 families: summary

Row counts are from your `test.csv` (1,176,851 rows in total).

Family	What it is	Rows in test
DDoS	Flood a victim from *many* machines at once: SYN, UDP, ICMP and HTTP floods, fragmented packets, and SlowLoris (keeps connections half-open so the server runs out of slots).	855,981 (73%)
DoS	The same kinds of floods, but from a *single* machine.	204,245 (17%)
Mirai	Floods generated by the Mirai botnet, malware that infects cheap IoT devices (GRE and plain-UDP floods).	66,749 (5.7%)
Recon	Looking around before attacking: find live hosts, open ports, the operating system, known vulnerabilities.	8,812 (0.7%)
Spoofing	Pretending to be someone else: fake ARP replies (man-in-the-middle) or fake DNS answers.	12,410 (1.1%)
Web-based	Attacks on web apps: SQL injection, command injection, XSS, browser hijacking, uploading malware or backdoors.	626 (0.05%)
BruteForce	Guessing passwords from a word list.	319 (0.03%)
Benign	Normal traffic, no attack.	27,709 (2.4%)

Takeaways

1. **Binary is easiest, 34-class is hardest.** For this project binary is enough; do 8-class if time allows.
2. **The data is very unbalanced.** DDoS and DoS are 90% of all rows, while Web-based and BruteForce have only a few hundred. The model will be weak on the small ones; use class weights or undersample DDoS.
3. **Your live demo** (hping3 SYN/UDP flood, nmap port scan) falls under **DDoS/DoS** and **Recon**.

The 46 features: what each column means

Key idea first. Each row is *averaged over a small window of consecutive packets*, it is not a single raw packet. So a column that is 0 or 1 for one packet becomes a fraction:

- `syn_flag_number = 0.8` means *80% of the packets in that window had the SYN flag set.*
- `TCP = 1.0` means *every* packet in the window was TCP.

Column	Meaning (plain English)
A. Timing and speed	
`flow_duration`	Time between the first and last packet of the flow.
`Duration`	**Misleading name:** it is actually the packet's **TTL** (time-to-live, i.e. how many router hops it has left).
`Rate`	Packets per second, measured over the window. **Very high during floods.** This is the only speed column worth using.
`Srate`	Meant to be the rate of *outgoing* packets. **In your data it is identical to `Rate` in every row** ⇒ useless, drop it.
`Drate`	Meant to be the rate of *incoming* (reply) packets. **Non-zero in only 74 of 1.18 million rows** ⇒ useless, drop it. (See "How the data was captured".)
`IAT`	Inter-arrival time: the gap since the previous packet.
B. TCP flags in the packet (fraction of window, 0 to 1)	
`syn_flag_number`	SYN = "let's open a connection". **High in SYN floods.**
`ack_flag_number`	ACK = "got it". Normal two-way traffic has lots of these.
`fin_flag_number`	FIN = "I'm closing the connection politely".
`rst_flag_number`	RST = "abort this connection". Closed ports reply with RST, so it rises during port scans.
`psh_flag_number`	PSH = "deliver this data to the app right now".
`ece_flag_number`, `cwr_flag_number`	Network congestion signals. Rare; usually 0.
C. Flag counters	
`syn_count, ack_count, fin_count, rst_count, urg_count`	How many packets in the flow had that flag so far (a running count, not a fraction).
D. Packet header and protocol number	
`Header_Length`	Size of the packet headers in bytes.
`Protocol Type`	IP protocol number: 6 = TCP, 17 = UDP, 1 = ICMP. Because it is averaged, it can be something like 6.4.
E. Which protocol is it? (fraction of window, 0 to 1)	
`HTTP, HTTPS`	Web traffic, plain and encrypted.
`DNS`	Name lookups ("what is the IP of google.com?").

Column	Meaning (plain English)
`Telnet, SSH`	Remote login: Telnet is unencrypted, SSH is encrypted.
`SMTP`	Sending e-mail.
`IRC`	Chat protocol, often used by botnets to receive commands.
`TCP`	Reliable, connection-based transport.
`UDP`	Fire-and-forget transport. **Equals 1 during UDP floods.**
`ICMP`	Ping and error messages. **Equals 1 during ping (ICMP) floods.**
`ARP`	"Who has this IP address?" on the local network. Abused in ARP spoofing.
`DHCP`	"Please give me an IP address."
`IPv`	Is the packet IP at all.
`LLC`	Low-level link-layer frame.

F. Packet size statistics (bytes)

`Tot sum`	Sum of the packet sizes.
`Min, Max`	Smallest and largest packet size.
`AVG, Std`	Average packet size and how much sizes vary.
`Tot size`	Average packet size. Floods use tiny, identical packets, so this is small and Std is near 0.
`Number`	Number of packets in the flow.

G. Incoming vs outgoing comparison

`Magnitue`	$\sqrt{\text{avg incoming size} + \text{avg outgoing size}}$: overall "how big" the traffic is. (The typo is in the dataset itself.)
`Radius`	$\sqrt{\text{incoming variance} + \text{outgoing variance}}$: how much sizes vary in both directions.
`Covariance`	Do incoming and outgoing sizes move together (normal conversation) or not (one-way flood)?
`Variance`	Ratio of incoming-size variance to outgoing-size variance.
`Weight`	(number of incoming packets) $\times$ (number of outgoing packets).

H. The answer

`label`	One of the 34 labels from the tree on page 1. This is what the model learns to predict.

Why CICDDoS2019 looked so confusing

That dataset was made with **CICFlowMeter**, where *one row = one connection* between two machines. Its column names use a few prefixes:

Prefix / word	Meaning
Fwd (forward)	Packets from the side that *started* the connection (client → server).
Bwd (backward)	The *replies* (server → client).
IAT	Inter-arrival time: the gap between packets.
Init_Win_bytes	TCP window size in the first packet.
Subflow	The connection cut into smaller pieces.
Bulk	Bursts of many packets sent back-to-back.
Active / Idle	How long the connection was busy vs quiet.

Examples: `Total Fwd Packets` = how many packets the client sent; `Bwd Packet Length Mean` = the average size of the server's replies. In a SYN flood the server barely replies, so most **Bwd** columns are 0.

CICIoT2023 mostly drops the Fwd/Bwd split and uses "incoming/outgoing" instead (`Srate`, `Drate`, `Magnitue`, `Radius`, ...).

Most useful columns for your demo attacks

Attack	Columns that give it away
SYN flood	`syn_flag_number`, `syn_count`, `Rate`, `TCP`
UDP flood	`UDP`, `Rate`, `Tot size`
Port scan	`rst_flag_number`, `rst_count`, `Number`, `Rate`